

Đồ án môn học

Xây dựng Thư viện sử dụng Templates

1 Giới thiệu

Đồ án môn học này yêu cầu mỗi cặp sinh viên cài đặt lại các cấu trúc dữ liệu và giải thuật đã học thành một thư viện có thể sử dụng được. Công việc này được chia thành hai vai trò:

- **Thành viên A - Phát triển thư viện:** cài đặt một thư viện C++ có thể sử dụng được, bao quát các cấu trúc dữ liệu và giải thuật đã được học trong học phần.
- **Thành viên B - Phát triển ứng dụng:** cài đặt một ứng dụng thực tế có ý nghĩa, ở đó sử dụng càng nhiều thành phần của thư viện càng tốt.

2 Cơ sở lý thuyết: C++ Templates

Trước khi cài đặt bất kỳ cấu trúc dữ liệu hay giải thuật nào, cả hai thành viên trong nhóm phải cần hiểu cách *C++ templates* hoạt động, vì toàn bộ thư viện sẽ được xây dựng dựa trên chúng.

2.1 Template là gì?

Template là một cơ chế cho phép các hàm hoặc struct/class làm việc với BẤT KỲ kiểu dữ liệu nào. Ví dụ, thay vì viết các phiên bản riêng biệt như `Stack<int>`, `Stack<double>`, và `Stack<string>`, ta chỉ cần định nghĩa một template, và trình biên dịch sẽ tự động sinh ra mã cụ thể tương ứng.

```
1  template <typename T>
2  struct Stack {
3      T* data;
4      int top;
5      int capacity;
6
7      Stack(int cap);
8      void push(const T& item);
9      T pop();
10 };
```

2.2 Một số ví dụ

Một **function template** hoạt động tương tự đối với các hàm độc lập:

```
1  template <typename T>
2  void mySwap(T& a, T& b)
3  {
4      T tmp = a;
5      a = b;
6      b = tmp;
7  }
```

Ví dụ về lời gọi:

```
1  int x = 3, y = 5;
2  mySwap(x, y);
3
4  double a = 3.14, b = 2.71;
5  mySwap(a, b);
```

Một ví dụ khác là một hàm template trả về giá trị lớn hơn trong hai giá trị:

```
1  template <typename T>
2  T myMax(T a, T b)
3  {
4      return (a > b) ? a : b;
5  }
```

Ví dụ về lời gọi:

```
1  int m = myMax(10, 20);
2  double n = myMax(4.5, 2.3);
```

Tất cả các lời gọi ở trên đều biên dịch thành công, vì trình biên dịch tự động suy ra được T từ các đối số trong mỗi lời gọi.

2.3 Cảnh báo khi cài đặt

Vì template được phân giải tại thời điểm biên dịch, **phần định nghĩa phải được cài đặt trong các tệp header** (hoặc được khởi tạo tường minh). Việc tách một lớp template thành .h và .cpp theo cách truyền thống sẽ gây ra lỗi liên kết. Cách tổ chức mã nguồn khuyến nghị là một tệp .hpp duy nhất chứa cả khai báo lẫn định nghĩa.

3 Thành viên A: Phát triển thư viện

Thành viên A sẽ cài đặt các cấu trúc dữ liệu và giải thuật đã được học dưới dạng một thư viện. Mỗi thành phần phải:

- Được cài đặt bằng C++ templates (tổng quát theo kiểu phần tử T).
- Có một chú thích ngắn phía trên mỗi hàm để giải thích cách sử dụng.

3.1 Danh sách liên kết

Cài đặt danh sách liên kết đơn. Tối thiểu cần hỗ trợ: `insertFront`, `insertBack`, `insertAt(index)`, `remove(value)`, `removeAt(index)`, `find(value)`, `size()`, `clear()`, và cả khả năng duyệt từ đầu hoặc từ cuối.

Struct	Mô tả	Tệp
<code>LinkedList<T></code>	Danh sách liên kết đơn	<code>LinkedList.hpp</code>

3.2 Ngăn xếp, hàng đợi và hàng đợi ưu tiên

Struct	Các hàm cốt lõi	Tệp
<code>Stack<T></code>	<code>push</code> , <code>pop</code> , <code>top</code> , <code>empty</code> , <code>size</code>	<code>Stack.hpp</code>
<code>Queue<T></code>	<code>enqueue</code> , <code>dequeue</code> , <code>front</code> , <code>empty</code> , <code>size</code>	<code>Queue.hpp</code>
<code>PriorityQueue<T></code>	<code>insert</code> , <code>extract</code> , <code>peek</code> , <code>empty</code> , <code>size</code>	<code>PriorityQueue.hpp</code>

3.3 Các thuật toán sắp xếp

Cài đặt các thuật toán sắp xếp sau dưới dạng *function templates* trong một header duy nhất `Algorithms.hpp` gồm `bubbleSort(arr, n)`, `selectionSort(arr, n)`, `insertionSort(arr, n)`, `heapSort(arr, n)`, `quickSort(arr, lo, hi)`, `mergeSort(arr, n)`.

Lưu ý quan trọng: Mỗi hàm sắp xếp cần phải nhận vào một mảng và một đối số bộ so sánh `Comp cmp = std::less<T>()` để cho phép sắp xếp theo bất kỳ thứ tự nào.

3.4 Các thuật toán tìm kiếm

Cài đặt các thuật toán tìm kiếm sau dưới dạng *function templates* trong một header duy nhất `Algorithms.hpp`: `linearSearch(arr, n, key)` và `binarySearch(arr, n, key)` (với tìm kiếm nhị phân thì yêu cầu mảng đã được sắp xếp sẵn).

3.5 Cấu trúc cây

Struct	Mô tả	Tệp
<code>BST<T></code>	Cây BST với <code>insert</code> , <code>remove</code> , <code>search</code> , duyệt theo tiền/trung/hậu thứ tự	<code>BST.hpp</code>
<code>AVL<T></code>	BST tự cân bằng; mở rộng BST với cơ chế tái cân bằng tự động	<code>AVL.hpp</code>

3.6 Bảng băm với chuỗi AVL

Cài đặt `HashTable<K, V>` bằng **separate chaining**, trong đó mỗi bucket là một `AVL<pair<K,V>` thay vì một danh sách liên kết thông thường.

Struct	Các hàm cốt lõi	Tệp
<code>HashTable<K,V></code>	<code>insert</code> , <code>remove</code> , <code>find</code> , <code>contains</code> , <code>size</code> , <code>clear</code>	<code>HashTable.hpp</code>

Lưu ý rằng không yêu cầu phải rehashing.

3.7 Cấu trúc tệp của thư viện

Ví dụ chi tiết về cách tổ chức thư mục của thư viện:

```
lib/
├── LinkedList.hpp
├── Stack.hpp
├── Queue.hpp
├── PriorityQueue.hpp
├── Algorithms.hpp
├── BST.hpp
├── AVL.hpp
└── HashTable.hpp
```

4 Thành viên B: Phát triển ứng dụng

Thành viên B đề xuất và cài đặt một **ứng dụng do tự chọn** đóng vai trò như là một trường hợp sử dụng thực tế không tầm thường cho thư viện. Ứng dụng phải thỏa mãn các tiêu chí sau:

1. **Tính nguyên bản:** Ý tưởng phải là của chính nhóm.
2. **Tính đúng đắn:** Mọi thao tác phải cho ra kết quả chính xác.
3. **Mức độ bao phủ:** Phải sử dụng *ít nhất 5* trong số 7 thành phần thư viện ở trong Mục 3. Việc sử dụng hời hợt hoặc vô nghĩa (ví dụ, chỉ thực hiện một thao tác tầm thường duy nhất trên một cấu trúc dữ liệu) sẽ không được tính.

4.1 Ví dụ về ý tưởng cho ứng dụng

Ý tưởng là **mở**, nhóm có thể chọn bất kỳ lĩnh vực nào, bao gồm nhưng không giới hạn ở:

- Một mạng xã hội đơn giản (tra cứu người dùng qua bảng băm).
- Một bộ lập lịch và quản lý việc cần làm (hàng đợi ưu tiên, danh sách liên kết, sắp xếp).
- Một trò chơi chạy trên console (quản lý kho đồ với nhiều cấu trúc khác nhau).
- Một hệ thống quản lý hồ sơ sinh viên (đánh chỉ mục BST, bảng băm, sắp xếp).
- Một bộ lập kế hoạch lộ trình hoặc điều hướng bản đồ (sử dụng hàng đợi ưu tiên cho Dijkstra).

4.2 Cấu trúc tệp của ứng dụng

Mã nguồn của ứng dụng PHẢI được **tách biệt** rõ ràng ra khỏi thư mục thư viện đã được cài đặt. Nhóm chỉ cần yêu cầu cài đặt và chạy demo ứng dụng sử dụng **giao diện dòng lệnh** là đủ rồi (không yêu cầu GUI).

Ví dụ chi tiết về cách tổ chức thư mục của ứng dụng:

```
app/  
├── main.cpp  
└── v.v...
```

Cấu trúc của nhóm không nhất thiết phải giống hệt như vậy, nhưng nên được tổ chức rõ ràng, đơn giản và chuyên nghiệp.

5 Hướng dẫn nộp bài

Vui lòng tuân theo các hướng dẫn nộp bài sau:

- Mã nguồn của nhóm cần phải nộp dưới dạng một tệp nén duy nhất (đuôi là .zip hoặc .rar) và được đặt tên theo định dạng **StudentID1_StudentID2**.
- Nếu tệp nén lớn hơn 20MB, hãy tải nó lên Google Drive và nộp tệp **.txt** có chứa liên kết. **Tuyệt đối không được phép chỉnh sửa sau thời hạn nộp bài.**

Ví dụ chi tiết về cách tổ chức thư mục:

```
StudentID1_StudentID2
├── lib/ (Thư mục mã nguồn đầy đủ của thư viện)
├── app/ (Thư mục mã nguồn đầy đủ của ứng dụng)
├── demo.txt (Chứa liên kết chia sẻ công khai đến video demo)
├── Makefile (Hướng dẫn biên dịch và chạy dự án)
├── README.md (Hướng dẫn cách biên dịch và chạy mã nguồn)
├── Report.pdf (Báo cáo lại quá trình cài đặt và kiểm thử)
└── v.v...
```

6 Thang điểm đánh giá

Tiêu chí	Mô tả	Điểm
Thư viện	Phần cài đặt cần bao quát các cấu trúc dữ liệu và giải thuật đã được học; sử dụng template đúng cách với mã nguồn gọn gàng và có ghi chú.	30
Ứng dụng	Có ý nghĩa, được cài đặt đúng và chạy bị không lỗi.	30
Báo cáo	Viết lại ngắn gọn quá trình cài đặt (không cần trình bày mã nguồn) và báo cáo lại những test case cũng như kết quả chạy được.	20
Video demo	Trình bày tổng quan ngắn gọn về thư viện đã cài đặt và demo ứng dụng từ đầu đến cuối.	20
		Tổng 100

7 Lưu ý

Vui lòng chú ý các lưu ý sau:

- Đây là đồ án **NHÓM**. Sinh viên được yêu cầu làm việc theo cặp đôi.
Tuy nhiên, nếu một sinh viên không thể tìm được bạn cùng nhóm, bạn ấy có thể hoàn thành đồ án này một mình.
- Thời gian thực hiện: khoảng 4 tuần.
- Các công cụ AI **không bị hạn chế**; tuy nhiên, sinh viên nên sử dụng một cách khôn ngoan. Giảng viên hướng dẫn có quyền thực hiện các buổi vấn đáp bổ sung với các nhóm ngẫu nhiên để đánh giá mức độ hiểu biết về đồ án.
- Bất kỳ hình thức đạo văn, thiếu trung thực, gian lận đều sẽ dẫn đến điểm 0 cho học phần.

Phụ lục: Tài liệu gợi ý nên đọc

- Cormen, T.H. et al. *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- Stroustrup, B. *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.
- cppreference.com: <https://en.cppreference.com/w/cpp/language/templates>
- ISO C++: <https://isocpp.org/wiki/faq/templates>

Hết.